Tutorial: Calculating RMU reliability for a reinforcement learning task, using hBayesDM

Giacomo Bignardi

2026-09-14

Claude code was used to prepare the initial version of this tutorial using Giacomo's other tutorials and code for analysing this task. This was then checked and edited by Giacomo.

Introduction

> **Note**
>
> This tutorial can be run and tested inside the docker image `bignardig/tidyverse461:v6`, which has all the required R packages pre-installed (note: `cmdstan` still needs to be installed separately – see Setup below). The `v6` image differs from the `v5` image used for the paper's analysis scripts only in that it ships the newer version of `gbtoolbox`, in which the `reliability()` function has been renamed `rmu()`. The old name still works (with a deprecation warning), so the analysis code is unaffected.
>
> You don't need docker to follow along, though – you can also run the code the usual way, in your own R installation. The code in Setup will install all the necessary R packages for you.

This is a brief walk-through of how to calculate reliability using relative measurement uncertainty (RMU) described in our pre-print, this time applied to a **reinforcement learning task** rather than a signal-detection model (see the go/no-go tutorial for that example).

Reinforcement learning models can be fit hierarchically in a Bayesian framework, and `hBayesDM` is a popular R package that ships ready-made Stan models for many common tasks, so you don't have to write your own Stan code. This tutorial shows:

1. How data from a "fluctuating bandit" reinforcement learning task looks.
2. How to fit a hierarchical Bayesian model to this data using `hBayesDM::bandit4arm_4par()`.
3. How to calculate RMU reliability **manually**, by pulling a matrix of posterior draws out of the fitted model yourself. This approach can be applied beyond `hBayesDM` models.
4. How to calculate RMU reliability **automatically**, by passing the fitted `hBayesDM` model object straight to `rmu()`.

In the folder `applied_example_pike/` I have more analyses, directly comparing RMU estimates to split-half and test-retest estimates. This tutorial will focus on estimating reliability using RMU, rather than providing a comprehensive workflow for fitting Bayesian models with best practices.

Data

We use the Fluctuating Bandit task (task A) from Pike, A. C., … Roiser, J. P. (2026). The recoverability, reliability, and generalizability of reward processing parameters and relation to mental health symptoms. Psychological Medicine, 56, e238. The data and Stan models are openly available on the Open Science Framework.

Participants completed this task at two time points, but far fewer participants returned for the second session (N = 115) than completed the first (N = 547). **In this tutorial we only use time 1 data, and only for the N = 115 participants who also completed time 2** (i.e. the smaller of the two time-1 sub-samples) – this matches the sample used for the test-retest/split-half comparisons in `applied_example_pike/`, and keeps the model small enough to fit quickly.

It is worth emphasising that the main benefit of RMU is that it does not need retest data - so in practice we would use the larger dataset to get a more accurate estimate.

Setup

Getting the code and data

The source file for this tutorial (`tutorial_calculating_rmu_bandit.qmd`) and the bandit data it uses (in `data/pike_2026/`) are both in the paper's GitHub repository, giac01/8_bayes_reliability. The easiest way to get everything is to clone the repository, which needs git to be installed (the download is around 120 MB, as the repository also contains the paper's simulation code and results):

```
git clone https://github.com/giac01/8_bayes_reliability.git
cd 8_bayes_reliability
```

If you'd rather not use git, you can download the same files as a zip archive from the repository page (the green **Code** button, then **Download ZIP**, or this direct link) and unzip it.

The tutorial `.qmd` files sit in the root folder of the repository, and the data paths in the code below are relative to that folder. If you open the `.qmd` file in RStudio and run the chunks or render it, the working directory is set to the repository root for you. If you're instead running the code in a plain R session, set the working directory to the repository root first, e.g. `setwd("~/8_bayes_reliability")`.

If you're using the docker image, start the container from inside the cloned folder, mounting it as the working directory. `cmdstan` is not included in the image, so the command below also mounts your `cmdstan` installation folder (`~/.cmdstan`, where `install_cmdstan()` puts it) into the container, at the location where `cmdstanr` will look for it – see Pointing cmdstanr at cmdstan below. For example, the following renders this tutorial:

```
docker run --rm -it --user rstudio \
  -v "$PWD":/home/rstudio/8_bayes_reliability \
  -v "$HOME/.cmdstan":/home/rstudio/.cmdstan \
  -w /home/rstudio/8_bayes_reliability \
  bignardig/tidyverse461:v6 \
  quarto render tutorial_calculating_rmu_bandit.qmd
```

Installing the required R packages

The code below checks for the R packages this tutorial needs and installs any that are missing. Two of them are not on CRAN:

- My package of miscellaneous R functions, `gbtoolbox`, which contains the `rmu()` function used to calculate RMU. It is installed from github; if you already have an older version installed, which has `reliability()` but no `rmu()`, it will be updated.
- `hBayesDM` **version 2.0 or later**, which fits its models via `cmdstanr`. The version on CRAN (1.x) uses `rstan` instead, and its fitted model objects are not supported by `rmu()`, so we install the development version from github.

```
# Install any missing CRAN packages (does nothing if they are all already
installed)
required_packages = c("tidyverse", "gt", "ggdist", "knitr", "remotes")
missing_packages           =      required_packages[!sapply(required_packages,
requireNamespace, quietly = TRUE)]
if (length(missing_packages) > 0) install.packages(missing_packages)

# Install (or update) gbtoolbox from github if rmu() isn't available
if   (!requireNamespace("gbtoolbox",   quietly   =   TRUE)   ||   !"rmu"   %in%
getNamespaceExports("gbtoolbox")) {
  remotes::install_github("giac01/gbtoolbox")
}

# Install hBayesDM >= 2.0 (cmdstanr backend) from github if it isn't available
if (!requireNamespace("hBayesDM", quietly = TRUE) || packageVersion("hBayesDM")
< "2.0") {
  remotes::install_github("CCS-Lab/hBayesDM", subdir = "R")
}
```

! Installing `cmdstanr` and `cmdstan`

`hBayesDM` (>= 2.0) fits its Stan models via `cmdstanr`, so unlike the `brms` tutorials there is no option to use `rstan` instead. `cmdstanr` is not on CRAN: install the R package from the Stan R-universe repository, then install `cmdstan` itself (this needs a working C++ toolchain):

```
install.packages("cmdstanr", repos = c("https://stan-dev.r-universe.dev",
getOption("repos")))
cmdstanr::check_cmdstan_toolchain(fix = TRUE)
cmdstanr::install_cmdstan()
```

See the cmdstanr installation guide if you run into problems.

Load packages

```
rm(list = ls(all.names = TRUE))
set.seed(1)

library(tidyverse)
library(hBayesDM)
library(cmdstanr)
library(gbtoolbox)
library(gt)

data_dir = file.path("data", "pike_2026")
```

Pointing cmdstanr at cmdstan

`cmdstanr` needs to know where `cmdstan` is installed. It finds it automatically if `cmdstan` was installed with `install_cmdstan()`, which puts it in `~/.cmdstan` (the newest version in that folder is used), or if the `CMDSTAN` environment variable points to it. If you installed `cmdstan` somewhere else, tell `cmdstanr` where it is with `set_cmdstan_path()`. Either way, the following should print the path to your `cmdstan` folder, and will give an error if `cmdstanr` can't find one:

```
# Only needed if cmdstan is in a non-standard location:
# cmdstanr::set_cmdstan_path("/path/to/your/cmdstan")

cmdstanr::cmdstan_path()
```

```
[1] "/home/rstudio/.cmdstan/cmdstan-2.39.0"
```

Load Data

`bandit.csv` contains every participant's time 1 data (N = 547); `bandit2.csv` contains time 2 data, only for the N = 115 participants who returned for a second session. We use the IDs in `bandit2.csv` purely to identify our smaller sub-sample, then keep only their time 1 rows from `bandit.csv`.

```
bandit_raw_t1 = read_csv(file.path(data_dir, "bandit.csv"),   show_col_types =
FALSE)
bandit_raw_t2 = read_csv(file.path(data_dir, "bandit2.csv"), show_col_types =
FALSE)

ids_retest_sample = bandit_raw_t2 |> distinct(id) |> arrange(id) |> pull(id)

bandit_raw_t1 = bandit_raw_t1 |>
  filter(id %in% ids_retest_sample) |>
  select(id, trial_nr, is_gain, is_loss, response)

n_trials_per_pps = count(bandit_raw_t1, id)$n

cat("Number of participants:", n_distinct(bandit_raw_t1$id), "\n")
```

```
Number of participants: 115
```

```
cat("Trials  per  participant:",  if  (length(unique(n_trials_per_pps))  ==  1)
unique(n_trials_per_pps) else paste(range(n_trials_per_pps), collapse = "-"),
"\n")
```

```
Trials per participant: 200
```

This is what the raw trial-level data looks like – one row per trial, for the columns we actually need:

```
bandit_raw_t1 |>
  slice(1:10) |>
  knitr::kable(caption = "First 10 rows of the raw time 1 data")
```

id	trial_nr	is_gain	is_loss	response
57	0	1	0	4
57	1	1	0	2
57	2	0	0	1
57	3	0	0	3

id	trial_nr	is_gain	is_loss	response
57	4	0	0	2
57	5	1	0	4
57	6	0	0	4
57	7	0	1	2
57	8	0	1	1
57	9	0	1	4

- `id`: participant ID
- `trial_nr`: trial number (0-indexed; 200 trials per participant)
- `is_gain`: whether a reward was delivered on that trial for the option chosen (1) or not (0)
- `is_loss`: whether a punishment was delivered on that trial for the option chosen (1) or not (0)
- `response`: which of the 4 bandit arms (1-4) the participant chose

The Fluctuating Bandit Task

On each trial, participants choose one of four "bandit" arms. Each arm can independently deliver a reward and/or a punishment, and the underlying reward/punishment probabilities of each arm drift over the course of the task, so participants have to keep learning and updating their beliefs rather than settling on one best option.

We will fit `hBayesDM::bandit4arm_4par()`, a hierarchical Rescorla-Wagner model with 4 free parameters per participant:

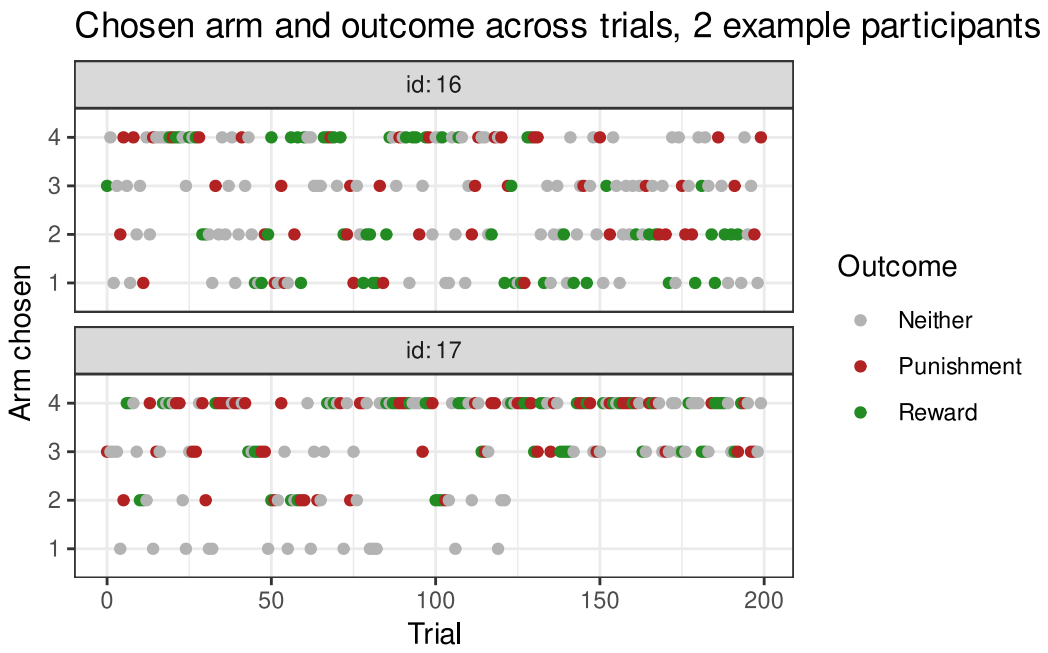
- `Arew` – reward learning rate
- `Apun` – punishment learning rate
- `R` – reward sensitivity
- `P` – punishment sensitivity

A full description of the model equations and the Stan code itself is in the appendix of the full applied example; we won't repeat it here.

A quick visualisation of the raw data, for a couple of participants, showing which arm was chosen on each trial and whether it paid out a reward or punishment:

```
bandit_raw_t1 |>
  filter(id %in% ids_retest_sample[1:2]) |>
  mutate(outcome = case_when(
    is_gain == 1 ~ "Reward",
    is_loss == 1 ~ "Punishment",
    TRUE         ~ "Neither"
  )) |>
  ggplot(aes(x = trial_nr, y = factor(response), color = outcome)) +
  geom_point(size = 1.5) +
```

```
  facet_wrap(~id, ncol = 1, labeller = label_both) +
    scale_color_manual(values  =  c("Reward"  =  "forestgreen",  "Punishment"  =
"firebrick", "Neither" = "grey70")) +
  labs(
    title = "Chosen arm and outcome across trials, 2 example participants",
    x = "Trial", y = "Arm chosen", color = "Outcome"
  ) +
  theme_bw()
```



Prepare Data for hBayesDM

`hBayesDM::bandit4arm_4par()` expects one row per trial, with columns `subjID`/`choice`/`gain`/`loss`, in contrast to the `id`/`trial_nr`/`is_gain`/`is_loss`/`response` columns in the raw data above. It has no `trial_nr` column of its own – it infers each participant's trial sequence purely from row order – so we arrange by participant and trial number, then rename the columns it needs:

```
bandit_df = bandit_raw_t1 |>
  arrange(id, trial_nr) |>
  group_by(id, trial_nr) |>
   slice_tail(n = 1) |>  # keep the last recorded row for any repeated trial
number (affects a handful of trials for one participant)
  ungroup() |>
  transmute(subjID = id, choice = response, gain = is_gain, loss = is_loss)
```

```r
bandit_df |>
  slice(1:10) |>
  knitr::kable(caption = "First 10 rows, hBayesDM-ready data")
```

subjID	choice	gain	loss
16	3	1	0
16	4	0	0
16	1	0	0
16	3	0	0
16	2	0	1
16	4	0	1
16	3	0	0
16	1	0	0
16	4	0	1
16	2	0	0

Fit the Model

We now fit `bandit4arm_4par()` to all 115 participants at once – it's a *hierarchical* model, so every participant's `Arew`/`Apun`/`R`/`P` estimate borrows statistical information from the group.

We use `niter = 3000` (`nwarmup = 1000` of which are warmup, leaving 2,000 post-warmup draws per chain), `nchain = 4`, `ncore = 4`, and `adapt_delta = 0.95`. This is a hard to sample model – expect this chunk to take several hours. In `applied_example_pike/` we used even more conservative settings (`niter = 8000`, `nwarmup = 3000`, `adapt_delta = 0.975`) and inspect the diagnostics in more detail.

```r
fit_bandit = bandit4arm_4par(
  data        = bandit_df,
  niter       = 3000,
  nwarmup     = 1000,
  nchain      = 4,
  ncore       = 4,
  adapt_delta = 0.95,
  seed        = 1
)

# cmdstanr fit objects lazily read draws/diagnostics from CSV files on disk the
first time
```

```
# they're requested. Since we're relying on knitr's cache to persist `fit_bandit`
across
# separate renders (i.e. separate R sessions, possibly after those CSV files no
longer exist),
# we force everything to be read into memory now -- the same thing cmdstanr's own
# fit$save_object() does before writing a fit to disk.
invisible(fit_bandit$fit$draws())
try(fit_bandit$fit$sampler_diagnostics(), silent = TRUE)
```

`fit_bandit` is an object of class `hBayesDM`, which bundles together several things we'll need below:

```
class(fit_bandit)
```

```
[1] "hBayesDM"
```

```
names(fit_bandit)
```

```
[1] "model"        "all_ind_pars" "par_vals"     "fit"          "raw_data"
```

- `fit_bandit$fit` is the underlying `cmdstanr` fit object (a `CmdStanMCMC`), which is what you'd normally interact with directly if you'd written your own Stan model.
- `fit_bandit$all_ind_pars` is a data frame of posterior-mean point estimates for each participant.
- `fit_bandit$par_vals` is a list containing every parameter's full posterior draws.

A quick convergence check

Before trusting any reliability estimates from this fit, we check for divergent transitions and look at Rhat/ESS for the group-level (`mu_pr`, `sigma`) and individual-level (`Arew`, `Apun`, `R`, `P`) parameters. Conventional thresholds are no divergent transitions, Rhat < 1.01, and ESS > 400:

```
d = fit_bandit$fit$diagnostic_summary()
cat("Divergent transitions:", sum(d$num_divergent), "\n")
```

```
Divergent transitions: 0
```

```
cat("Max treedepth hits:   ", sum(d$num_max_treedepth), "\n")
```

```
Max treedepth hits:    0
```

```
cat("Min E-BFMI:           ", round(min(d$ebfmi), 3), "\n")
```

```
Min E-BFMI:            0.664
```

```
fit_bandit$fit$summary(variables = c("mu_pr", "sigma", "Arew", "Apun", "R",
"P")) |>
  mutate(parameter = str_extract(variable, "^[^\\[]+")) |>
  group_by(parameter) |>
  summarise(
    max_rhat     = max(rhat, na.rm = TRUE),
    min_ess_bulk = min(ess_bulk, na.rm = TRUE),
    min_ess_tail = min(ess_tail, na.rm = TRUE),
    .groups = "drop"
  ) |>
  gt() |>
  fmt_number(columns = max_rhat, decimals = 3) |>
  fmt_number(columns = c(min_ess_bulk, min_ess_tail), decimals = 0)
```

parameter	max_rhat	min_ess_bulk	min_ess_tail
Apun	1.002	2,809	1,733
Arew	1.003	3,555	1,488
P	1.002	2,599	2,036
R	1.002	3,402	1,559
mu_pr	1.006	545	1,163
sigma	1.004	1,100	2,123

If any divergences show up or Rhat/ESS still look poor at these settings, try increasing `adapt_delta` further and running more warmup and sampling iterations.

Reliability Estimation

RMU reliability only needs the posterior draws for each participant's parameter, from a single model fit – there's no need for retest data or split-half models. RMU is described in the go/no-go tutorial and in the pre-print.

Manually: pulling a matrix of posterior draws

`rmu()`'s default method just needs a matrix with one row per participant and one column per posterior draw. We can pull this straight out of the `cmdstanr` fit object, for a single parameter (here, `Arew`, the reward learning rate).

`rmu()` calculates RMU by randomly splitting each participant's posterior draws into two halves and correlating them, repeated many times to build up a posterior distribution for reliability itself (see the go/no-go tutorial for the full mechanics). Because we only have a finite number of posterior draws to split, *which* random split we happen to get will nudge the resulting point estimate and HDCI slightly – rerunning `rmu()` on the exact same draws will give you a very similar, but not identical, answer each time. We set a seed before each call below so the results are reproducible, and so that the manual and automatic approaches (which use the same draws, in the same parameter order) give identical estimates rather than just very similar ones.

```
set.seed(42)

Arew_draws = fit_bandit$fit$draws(variables = "Arew", format = "matrix") |>  #
draws x participants
  t()  # transpose so participants are on rows, draws are on columns

dim(Arew_draws)  # 115 participants x 8000 posterior draws (4 chains x 2000 post-
warmup draws)
```

```
[1]  115 8000
```

```
rel_Arew_manual = rmu(Arew_draws)

print(rel_Arew_manual)
```

```
Reliability (Relative Measurement Uncertainty)
  Number of subjects:              115
  Number of input posterior draws: 8000
  Number of RMU posterior draws:   4000

  RMU estimate: 0.854
  Posterior SD: 0.037
  95% HDCI:     [0.780, 0.921]

Access full results with $hdci and $reliability_posterior_draws
```

We can repeat this for each of the 4 individual-level parameters in the model:

```
subj_pars = c("Arew", "Apun", "R", "P")

set.seed(42)
```

```r
rel_manual = map_dfr(subj_pars, function(par){
  draws = fit_bandit$fit$draws(variables = par, format = "matrix") |> t()
  rel   = rmu(draws)
  tibble(
    parameter = par,
    rmu       = rel$hdci$rmu_estimate,
    lower     = rel$hdci$hdci_lowerbound,
    upper     = rel$hdci$hdci_upperbound
  )
})

rel_manual |>
  gt() |>
  fmt_number(columns = c(rmu, lower, upper), decimals = 3)
```

parameter	rmu	lower	upper
Arew	0.854	0.780	0.921
Apun	0.482	0.333	0.624
R	0.860	0.788	0.928
P	0.856	0.781	0.922

Automatically: passing the fitted hBayesDM model directly

`gbtoolbox::rmu()` is a generic function – if you pass it an object of class `hBayesDM` (rather than a plain matrix), it dispatches to `rmu.hBayesDM()`, which pulls out the posterior draws for you and calculates RMU reliability for every individual-level parameter in the model automatically (by default; you can also pass `parameter = c("Arew", "R")` etc. to only calculate reliability for specific parameters). The same works for models fit with `brms` (objects of class `brmsfit`) – see the go/no-go tutorial for an example.

```r
set.seed(42)

rel_auto = rmu(fit_bandit, parameter = subj_pars)
```

```
Reliability (Relative Measurement Uncertainty) Estimates
Posterior Mean & 95% HDCI
  Number of subjects:              115
  Number of input posterior draws: 8000
  Number of RMU posterior draws:   4000
```

```
     rmu_estimate hdci_lowerbound hdci_upperbound
Arew        0.854           0.780           0.921
Apun        0.482           0.333           0.624
R           0.860           0.788           0.928
P           0.856           0.781           0.922
```

This prints a summary table directly, and also returns a named list (one element per parameter, each with the same `$hdci`/`$reliability_posterior_draws` structure as the manual approach above):

```
rel_auto$Arew$hdci
```

```
  rmu_estimate hdci_lowerbound hdci_upperbound .width .point .interval
1    0.8535059       0.7796418       0.9205787   0.95   mean      hdci
```

How to calculate RMU on rescaled parameters

RMU (like an ICC) is not invariant to nonlinear transformations. It is important to calculate RMU using the same scaling that will be used in the main analysis.

The analysis model actually creates the sensitivity and learning rate parameters by transforming normally distributed variables $z_i^r, z_i^p, z_i^P, z_i^R$ through the standard normal cumulative distribution function $\Phi(\cdot)$ (`pnorm()` in R), which maps them onto $(0, 1)$ – see the hierarchical priors section of the full applied example for the Stan code this comes from.

$$\alpha_i^r = \Phi(\mu_1 + \sigma_1 z_i^r), \quad \alpha_i^p = \Phi(\mu_2 + \sigma_2 z_i^p), \quad R_i = 30 \cdot \Phi(\mu_3 + \sigma_3 z_i^R), \quad P_i = 30 \cdot \Phi(\mu_4 + \sigma_4 z_i^P)$$

We can run that transformation in reverse, dividing the sensitivities (`R`, `P`) by 30 to bring them back onto $(0, 1)$, then applying the inverse of Φ (`qnorm()`) to every posterior draw – to recover each participant's posterior draws on the original unconstrained ("probit") scale, and check whether RMU computed on that scale differs from RMU computed on the native $(0, 1)/(0, 30)$ scale we used above.

Instead of using this transformation, you can use whatever function is appropriate in your case (e.g., log-transform, logit).

Scaling the posterior draws

We pull each parameter's draws out as its own participants x draws matrix, then transform each one by hand. Writing this out one parameter at a time, rather than wrapping it in a helper function, makes it easy to see exactly what happens to each parameter:

```
eps = 1e-6  # a handful of draws hit exactly 0 or 1 (Phi_approx saturation) ->
qnorm(0/1) = +-Inf
```

```
# native scale: participants x draws matrices, straight off the fitted model
draws_Arew  =  unclass(fit_bandit$fit$draws(variables  =  "Arew",  format  =
"matrix")) |> t()
draws_Apun  =  unclass(fit_bandit$fit$draws(variables  =  "Apun",  format  =
"matrix")) |> t()
draws_R   = unclass(fit_bandit$fit$draws(variables = "R",   format = "matrix"))
|> t()
draws_P   = unclass(fit_bandit$fit$draws(variables = "P",   format = "matrix"))
|> t()

# probit (z) scale: invert Phi() by hand for each parameter
# Arew, Apun are already on (0, 1); R, P need dividing by 30 first
draws_Arew_z = qnorm(pmin(pmax(draws_Arew,      eps), 1 - eps))
draws_Apun_z = qnorm(pmin(pmax(draws_Apun,      eps), 1 - eps))
draws_R_z    = qnorm(pmin(pmax(draws_R / 30,    eps), 1 - eps))
draws_P_z    = qnorm(pmin(pmax(draws_P / 30,    eps), 1 - eps))
```

Posterior densities: native vs. probit-transformed scale

To see what this transform does, we plot per-participant posterior densities before and after, for a subset of participants. `matrix_to_long()` below is purely for reshaping a matrix into the long format `ggplot()` wants – it doesn't do any rescaling itself:

```
ids      = fit_bandit$all_ind_pars$subjID
plot_ids = ids[1:30]  # a subset, for a readable plot

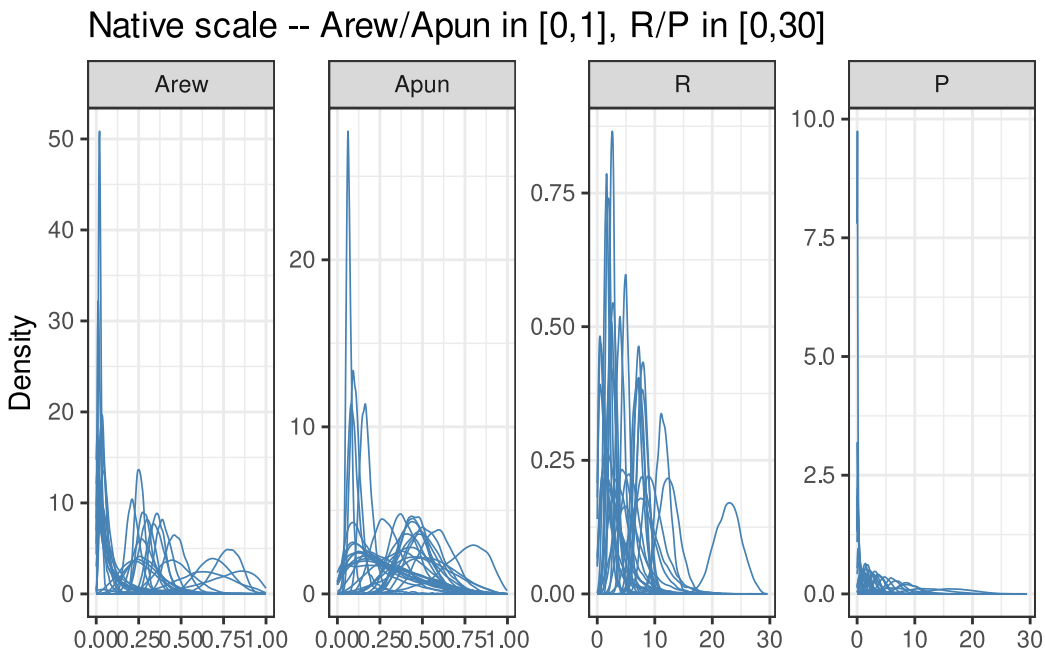
matrix_to_long = function(m, parameter_name) {
  rownames(m) = as.character(ids)  # as.character() so we subset rows by id
below, not by row position
  colnames(m) = paste0("draw", seq_len(ncol(m)))
  m[as.character(plot_ids), , drop = FALSE] |>
    as_tibble(rownames = "id") |>
    pivot_longer(-id, names_to = "draw", values_to = "value") |>
    mutate(parameter = parameter_name)
}

draws_native_long = bind_rows(
  matrix_to_long(draws_Arew, "Arew"),
  matrix_to_long(draws_Apun, "Apun"),
  matrix_to_long(draws_R,    "R"),
  matrix_to_long(draws_P,    "P")
) |>
  mutate(parameter = factor(parameter, levels = subj_pars))
```

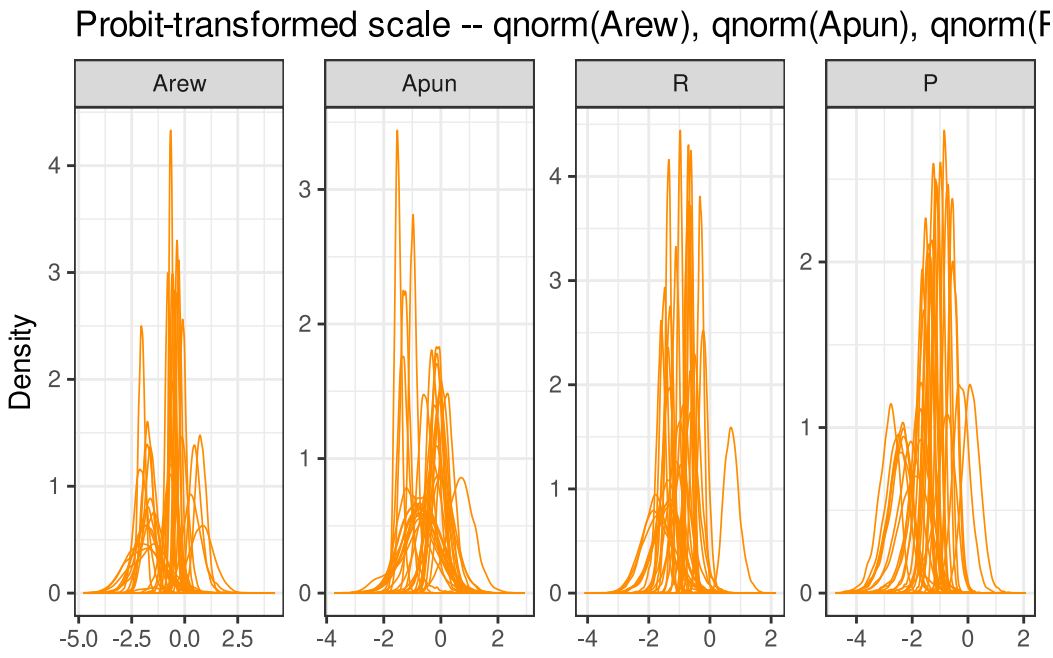
```
draws_probit_long = bind_rows(
  matrix_to_long(draws_Arew_z, "Arew"),
  matrix_to_long(draws_Apun_z, "Apun"),
  matrix_to_long(draws_R_z,    "R"),
  matrix_to_long(draws_P_z,    "P")
) |>
  mutate(parameter = factor(parameter, levels = subj_pars))

plot_subject_densities = function(draws_long, title, color) {
  draws_long |>
    ggplot(aes(x = value, group = id)) +
    geom_density(alpha = 0.1, linewidth = 0.3, color = color) +
    facet_wrap(~parameter, scales = "free", nrow = 1) +
    labs(title = title, x = NULL, y = "Density") +
    theme_bw()
}

plot_subject_densities(draws_native_long, "Native scale -- Arew/Apun in [0,1],
R/P in [0,30]", "steelblue")
```




```
plot_subject_densities(draws_probit_long,    "Probit-transformed    scale    --
qnorm(Arew), qnorm(Apun), qnorm(R/30), qnorm(P/30)", "darkorange")
```



Same participants, same underlying posterior – just a different (monotonic, one-to-one) scale for each draw.

RMU on the probit-transformed scale

`rmu()` doesn't care what scale a matrix is on – we just pass it each transformed matrix in turn:

```
set.seed(42)

rel_Arew_z = rmu(draws_Arew_z)
rel_Apun_z = rmu(draws_Apun_z)
rel_R_z    = rmu(draws_R_z)
rel_P_z    = rmu(draws_P_z)

rel_manual_transformed = tibble(
  parameter = c("Arew", "Apun", "R", "P"),
  rmu       = c(rel_Arew_z$hdci$rmu_estimate,    rel_Apun_z$hdci$rmu_estimate,
rel_R_z$hdci$rmu_estimate,    rel_P_z$hdci$rmu_estimate),
 lower    = c(rel_Arew_z$hdci$hdci_lowerbound, rel_Apun_z$hdci$hdci_lowerbound,
rel_R_z$hdci$hdci_lowerbound, rel_P_z$hdci$hdci_lowerbound),
 upper    = c(rel_Arew_z$hdci$hdci_upperbound, rel_Apun_z$hdci$hdci_upperbound,
rel_R_z$hdci$hdci_upperbound, rel_P_z$hdci$hdci_upperbound)
)

rel_manual_transformed |>
```

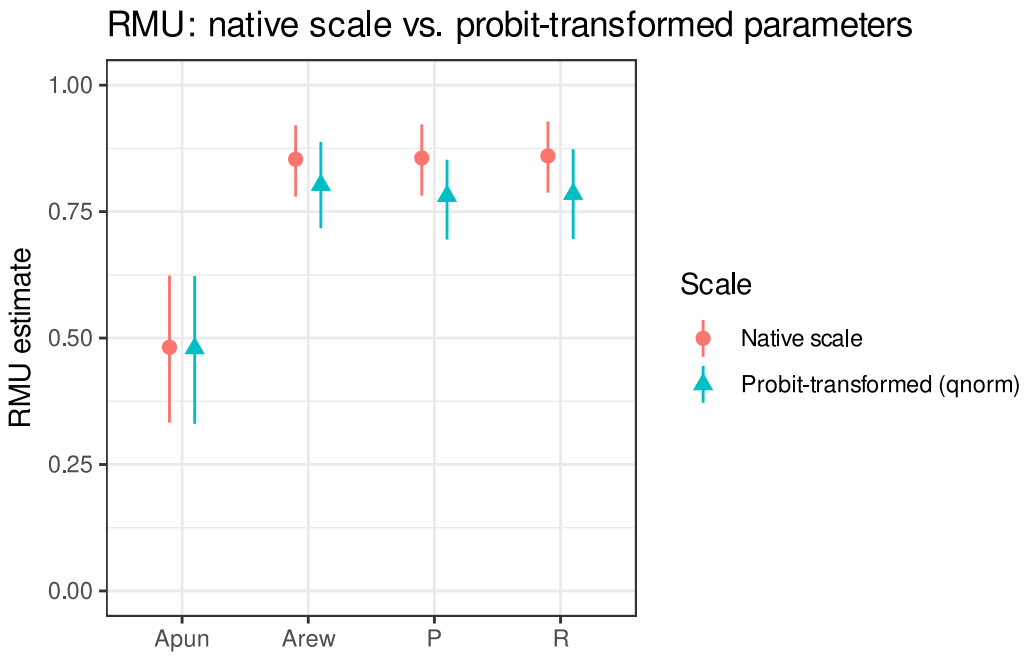
```
  gt() |>
  fmt_number(columns = c(rmu, lower, upper), decimals = 3)
```

parameter	rmu	lower	upper
Arew	0.803	0.717	0.888
Apun	0.480	0.330	0.623
R	0.785	0.696	0.873
P	0.781	0.695	0.853

Comparing native-scale and probit-transformed RMU

```
rmu_scale_comparison = bind_rows(
  rel_manual             |> mutate(scale = "Native scale"),
  rel_manual_transformed |> mutate(scale = "Probit-transformed (qnorm)")
)

rmu_scale_comparison |>
  ggplot(aes(x = parameter, y = rmu, color = scale, shape = scale)) +
      geom_pointrange(aes(ymin   =   lower,   ymax   =   upper),   position   =
position_dodge(width = 0.4)) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(
    x = NULL, y = "RMU estimate", color = "Scale", shape = "Scale",
    title = "RMU: native scale vs. probit-transformed parameters"
  ) +
  theme_bw()
```



Here they line up closely, but that won't always be true – when RMU differs noticeably by scale, it's a sign that "how reliable is this parameter" doesn't have one scale-free answer, and the scale you report reliability on should be the one you'll actually use in downstream analyses. For a full write-up of this same check on the much larger full T1 sample (N = 547), including tables of the posterior probability that each parameter's reliability falls in the "poor" (< .6), "okay" (.6-.8) or "good" (> .8) range, see the applied example.

Empirical reliability

Next we will compare RMU to the empirical reliability method (Du Toit, 2003), which is another approach to calculating reliability for Bayesian or maximum-likelihood models. In our pre-print, we show that point-estimates from RMU and empirical reliability when estimated from the same posterior draws should be identical in large samples.

Take a single parameter as an example, say the reward learning rate `Arew`. For each participant p, the fitted model gives us a point estimate, $\hat{\theta}_p$ (`Arew`'s posterior mean for that participant), and a measure of how uncertain that estimate is, $\hat{V}_p$ (`Arew`'s posterior *variance* for that participant – i.e. the squared posterior SD of their `Arew` draws). Across all $N = 115$ participants, this gives us two vectors, $\hat{\theta} = \left\{\hat{\theta}_1, \hat{\theta}_2, ..., \hat{\theta}_N\right\}$ and $\hat{V} = \left\{\hat{V}_1, \hat{V}_2, ..., \hat{V}_N\right\}$ – exactly the `mean` and (squared) `sd` columns that `fit_bandit$fit$summary()` reports for each participant.

Empirical reliability is essentially the classical reliability equation, $\sigma^2_{\text{truescores}}/(\sigma^2_{\text{truescores}} + \sigma^2_{\text{error}})$ (Revelle & Condon, 2018), built from these sample quantities: the variance of the point estimates across participants stands in for true-score variance, and the average of each participant's own posterior variance stands in for error variance:

$$\text{Empirical Reliability} = \frac{\widehat{\text{Var}}(\hat{\theta})}{\widehat{\text{Var}}(\hat{\theta}) + \sum_{p=1}^{N} \hat{V}_p / N}$$

Calculating empirical reliability

Unlike RMU, empirical reliability doesn't need the full matrix of posterior draws – just each participant's posterior mean ($\hat{\theta}_p$ above) and posterior variance ($\hat{V}_p$ above), which `cmdstanr`'s `summary()` gives us directly:

```
empirical_reliability = function(fit, par) {
  s          = fit$fit$summary(variables = par)  # one row per participant: posterior mean & sd
  theta_hat = s$mean  # each participant's point estimate
  V_hat      = s$sd^2  # each participant's posterior variance (sampling/measurement error)
  var(theta_hat) / (var(theta_hat) + mean(V_hat))
}

rel_empirical = map_dfr(subj_pars, function(par) {
           tibble(parameter     =     par,     empirical_reliability     = empirical_reliability(fit_bandit, par))
})

rel_empirical |>
  gt() |>
  fmt_number(columns = empirical_reliability, decimals = 3)
```

parameter	empirical_reliability
Arew	0.853
Apun	0.478
R	0.856
P	0.852

Comparing RMU and empirical reliability

```
rel_comparison = rel_manual |>
  select(parameter, rmu) |>
  left_join(rel_empirical, by = "parameter") |>
  mutate(difference = rmu - empirical_reliability)

rel_comparison |>
```

```
  gt() |>
  fmt_number(columns = c(rmu, empirical_reliability, difference), decimals = 3)
```

parameter	rmu	empirical_reliability	difference
Arew	0.854	0.853	0.001
Apun	0.482	0.478	0.004
R	0.860	0.856	0.004
P	0.856	0.852	0.005

The two methods agree closely here. But note the asymmetry: RMU comes with the `lower`/`upper` HDCI columns we calculated above, while empirical reliability gives us only a single point estimate – so when the two methods *do* disagree, there's no way to tell from empirical reliability alone whether that reflects genuine disagreement or just falls within RMU's own uncertainty.

Wrap-up

Check the convergence diagnostics above before reading much into the reliability estimates – if Rhat/ESS look healthy and there are no divergent transitions, the numbers above are a reasonable single-fit estimate of RMU reliability for this task and sample. For a more thoroughly-checked version of this exact model on this exact data (checked against test-retest and split-half reliability, and against a fit to the full N = 547 sample), see `applied_example_pike/02_02_pike_2026_bandit.qmd`.

The key takeaway for your own models: whichever way you fit a hierarchical Bayesian model, if you can get a matrix of posterior draws with participants on the rows and draws on the columns, you can calculate RMU reliability with `rmu()` – and if your model was fit with `hBayesDM` or `brms`, you can skip the matrix-pulling step entirely and just call `rmu()` on the fitted model object.

References

Du Toit, M. (Ed.). (2003). *IRT from SSI: BILOG-MG, MULTILOG, PARSCALE, TESTFACT*. Scientific Software International.

Pike, A. C., … Roiser, J. P. (2026). The recoverability, reliability, and generalizability of reward processing parameters and relation to mental health symptoms. *Psychological Medicine, 56*, e238.

Revelle, W., & Condon, D. M. (2018). Reliability. In P. Irwing, T. Booth, & D. J. Hughes (Eds.), *The Wiley handbook of psychometric testing: A multidisciplinary reference on survey, scale and test development* (1st ed.). John Wiley & Sons.

Session Info

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.4 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK:    /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.26.so;
LAPACK version 3.12.0

locale:
 [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
 [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
 [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
 [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
 [9] LC_ADDRESS=C               LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C

time zone: Etc/UTC
tzcode source: system (glibc)

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] gt_1.3.0            gbtoolbox_0.0.4     cmdstanr_0.9.0
 [4] hBayesDM_2.0.0.9000 lubridate_1.9.5     forcats_1.0.1
 [7] stringr_1.6.0       dplyr_1.2.1         purrr_1.2.2
[10] readr_2.2.0         tidyr_1.3.2         tibble_3.3.1
[13] ggplot2_4.0.3       tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] gtable_0.3.6         tensorA_0.36.2.1     xfun_0.59
 [4] remotes_2.5.0        processx_3.9.0       tzdb_0.5.0
 [7] vctrs_0.7.3          tools_4.6.1          ps_1.9.3
[10] generics_0.1.4       parallel_4.6.1       pkgconfig_2.0.3
[13] data.table_1.18.4    checkmate_2.3.4      RColorBrewer_1.1-3
[16] S7_0.2.2             distributional_0.8.1 lifecycle_1.0.5
[19] compiler_4.6.1       farver_2.1.2         codetools_0.2-20
[22] sass_0.4.10          htmltools_0.5.9      bayesplot_1.15.0
[25] yaml_2.3.12          pillar_1.11.1        crayon_1.5.3
[28] abind_1.4-8          posterior_1.7.0      tidyselect_1.2.1
[31] digest_0.6.39        stringi_1.8.7        labeling_0.4.3
[34] fastmap_1.2.0        grid_4.6.1           cli_3.6.6
```

```
[37] magrittr_2.0.5       loo_2.10.0           withr_3.0.3
[40] scales_1.4.0         backports_1.5.1      bit64_4.8.2
[43] timechange_0.4.0     rmarkdown_2.31       matrixStats_1.5.0
[46] bit_4.6.0            otel_0.2.0           hms_1.1.4
[49] evaluate_1.0.5       knitr_1.51           ggdist_3.3.3
[52] rlang_1.3.0          Rcpp_1.1.2           glue_1.8.1
[55] xml2_1.6.0           vroom_1.7.1          jsonlite_2.0.0
[58] R6_2.6.1             fs_2.1.0
```